

From M-P Neuron to Transformer: A Minimal Mathematical Path with Intuition-First Pedagogy for Deep Learning Education

Fujin Xie

Abstract

Deep learning education faces a fundamental tension: pure mathematics textbooks (e.g., [Goodfellow et al., 2016]) are rigorous but detached from practice, while engineering-oriented books (e.g., [Zhang et al., 2023]) teach code without building mathematical intuition. Content analyses of introductory textbooks reveal that most lack a coherent “conceptual thread,” appearing instead as a patchwork of disconnected techniques [Sulmont et al., 2019]. This paper presents a curriculum design framework that simultaneously addresses both gaps through two synergistic contributions.

First, we propose the **Intuition-First Triad**, a three-layer pedagogical model—intuition → formula → code—grounded in Bruner’s three modes of representation [Bruner, 1966], Sweller’s cognitive load theory [Sweller, 1988], and Tall & Vinner’s concept image theory [Tall & Vinner, 1981]. Second, we define a **Minimal Mathematical Path** from the M-P neuron to the Transformer, where each new concept emerges naturally as a solution to the limitations of the preceding one, forming a causally chained directed acyclic graph (DAG) of 22 core concepts across 10 transitional steps. We distill these principles into five transferable design principles for reuse in other AI education contexts.

The framework is instantiated in a complete open-source textbook, *Mathematics of AI Neural Networks* [Xie, 2026], available on GitHub, comprising 9 chapters with 35+ code files and 35+ visualizations. Evaluations include a DAG-based path completeness proof, a cognitive load analysis of symbol introduction curves against three major textbooks, and a qualitative comparison with existing pedagogical frameworks. Results demonstrate that the proposed framework fills a critical gap in deep learning education by unifying mathematical rigor, code verification, and pedagogical intentionality into a single, coherent design.

Keywords: deep learning education, curriculum design, intuition-first pedagogy, minimal mathematical path, cognitive load theory

1. Introduction

1.1 Motivation

The past decade has witnessed an explosive growth in demand for AI talent across industry and academia. Yet the pedagogical infrastructure for training this workforce remains fragmentary. Instructors and self-learners alike face a recurring dilemma when choosing learning materials: should they start with a mathematically rigorous text like Goodfellow, Bengio, and Courville’s *Deep Learning* [Goodfellow et al., 2016]—comprehensive but devoid of executable code and pedagogical scaffolding—or an engineering-oriented resource like *Dive into Deep Learning* [Zhang et al., 2023]—interactive and code-rich but often at the expense of conceptual depth?

This “math versus code” tension is not merely a matter of personal preference; it reflects a structural gap in how deep learning is currently taught. Sulmont et al. [2019] analyzed introductory deep learning textbooks and found that most present content as a “patchwork” of isolated topics—linear regression, then multi-layer perceptrons, then CNNs, then RNNs—without a unifying mathematical narrative that explains *why* each concept follows from the previous one. While mixed approaches combining theory and practice are generally recognized as most effective, no unified framework exists to guide their design.

Learners either understand the mathematics but cannot translate it into working code, or can run code but lack the conceptual foundation to debug, extend, or reason about their models—a gap that our framework directly addresses.

To address these challenges, we developed an open-source textbook, *Mathematics of AI Neural Networks* [Xie, 2026], which implements the proposed framework. The book is freely available on GitHub and serves as the concrete instantiation of the design principles discussed in this paper. This paper directly addresses the math-code gap by presenting the pedagogical framework underlying this textbook.

1.2 Research Gaps

Based on our comprehensive literature review, we identify five specific research gaps in current deep learning education:

#	Gap	Evidence
G1	Lack of a minimal, self-consistent knowledge path from fundamentals to modern architectures	Sulmont et al. [2019] found textbooks are “technical patchworks”
G2	No systematic intuition-first pedagogical framework for deep learning	Existing frameworks lack a structured intuition→formula→code progression

#	Gap	Evidence
G3	The “impossible triangle”—no textbook simultaneously achieves mathematical rigor, code verification, and pedagogical depth	Our textbook comparison matrix
G4	Course design papers rarely produce transferable design principles	Most papers are “course reports” rather than “design frameworks”
G5	No established method for evaluating curriculum coherence beyond credit hours	Existing evaluation is based on hours/topics, not dependency graphs

1.3 Contributions

This paper makes the following contributions:

- **C1:** We propose a **Minimal Mathematical Path** from the M-P neuron to the Transformer, with 10 causally chained transitions forming a directed acyclic graph (DAG) of 22 core concepts (§3).
- **C2:** We propose the **Intuition-First Triad**, a three-layer pedagogical model (intuition → formula → code) grounded in established cognitive theory, along with five transferable design principles (§2).
- **C3:** We instantiate the framework in a complete open-source textbook, *Mathematics of AI Neural Networks* [Xie, 2026], available on GitHub, with 9 chapters, 35+ code files, and 35+ visualizations (§4).
- **C4:** We evaluate the framework through DAG-based coherence analysis, cognitive load analysis via symbol introduction curves, and qualitative comparison with existing pedagogical frameworks (§5).

1.4 Paper Structure

Section 2 presents the Intuition-First Pedagogy, including its theoretical grounding and five design principles. Section 3 describes the Minimal Mathematical Path and demonstrates its mathematical self-consistency. Section 4 provides a detailed case study of the 9-chapter textbook implementation. Section 5 evaluates the framework through multiple analytical lenses. Section 6 discusses limitations, generalizability, and future work. Section 7 concludes.

2. The Intuition-First Pedagogy

This section presents the methodological core of our framework—not *what* we teach, but *why* we teach it that way.

2.1 Three Diagnosed Problems in Current DL Education

Our review of existing textbooks and educational research identifies three recurring pedagogical pathologies:

P1: Symbol Overload. Learners are introduced to an overwhelming number of mathematical symbols early in their journey. For instance, Goodfellow et al. [2016] introduces over 50 distinct mathematical symbols in the first two chapters alone, before any code is written.

P2: Code-First Fallacy. Engineering-oriented resources like *Dive into Deep Learning* [Zhang et al., 2023] adopt a “code-first” approach where learners execute Jupyter notebooks without necessarily understanding the underlying mathematics, producing practitioners who can “run code but cannot reason.”

P3: Path Discontinuity. Concepts are presented as independent modules rather than a causally connected chain. Sulmont et al. [2019] found that introductory textbooks typically jump directly from linear regression to multi-layer perceptrons without explaining why—the “M-P neuron → chain rule → gradient descent → backpropagation” narrative is missing, leaving learners unable to see deep learning as a unified mathematical framework.

2.2 Theoretical Grounding

Our pedagogy is not invented ad hoc but synthesized from six established cognitive and educational theories:

Theory	Source	Core Claim	Our Application
Three Modes of Representation	Bruner [1966]	Knowledge is represented through <i>enactive</i> (action), <i>iconic</i> (image), and <i>symbolic</i> (symbol) modes	Code layer (enactive) → Intuition layer (iconic) → Formula layer (symbolic), in reverse order
Cognitive Load Theory	Sweller [1988]	Instruction should minimize extraneous cognitive load	DP1 (Minimal Symbol) + DP4 (Progressive Abstraction)
Dual Coding Theory	Paivio [1986]	Verbal + visual channels enhance learning	Intuitive analogy (verbal) + visualization (visual) used simultaneously
Concept Image	Tall & Vinner [1981]		Intuition always precedes formula

Theory	Source	Core Claim	Our Application
		Learners need a conceptual image <i>before</i> formal definition	
Multimedia Learning	Mayer [2001]	Text + graphics + interactivity outperforms any single modality	Five-stage chapter design integrating all three
Threshold Concepts	Meyer & Land [2003]	Some concepts transform disciplinary understanding once grasped	Chain rule, δ recursion, convolution, attention as threshold nodes

Key Insight. Our triad is a *reverse application* of Bruner’s three modes. Bruner’s developmental path moves from enactive (infants touching objects) → iconic (children using images) → symbolic (adults using language). However, adult learners in our target audience already possess strong symbolic reasoning skills. They do not need to “start from code”; they need to start from *iconic representations* (intuitive analogies and visualizations) that anchor abstract concepts in familiar mental models, then formalize with symbols, and finally validate with code. This reversal is the pedagogical innovation that distinguishes our approach.

2.3 The Intuition-First Triad

The triad is a three-layer pedagogical structure applied to every concept in the curriculum. We illustrate it here using backpropagation (Chapter 5 of our textbook [Xie, 2026]), the conceptual heart of the course.

Layer 1: Intuition (Iconic). Each concept is first introduced through an intuitive analogy and a visualization that establish a mental model before any mathematics is presented. For backpropagation, the core intuition is: “Error propagates backward like dominoes—each δ (delta) represents a neuron’s share of responsibility for the final error.” A visualization animates the propagation of δ values from the output layer back through the hidden layers, with color intensity proportional to error magnitude. This layer draws on Tall & Vinner’s [1981] concept image theory, which emphasizes that learners need a qualitative understanding *before* engaging with formal definitions.

Layer 2: Formula (Symbolic). Once the mental model is established, the concept is formalized using minimal mathematical notation. For backpropagation, the δ recursion formula is presented with full annotation: $\delta^j = \delta^k \cdot w^{kj} \cdot f'(u^j)$, where δ^j denotes the error at layer j , w^{kj} represents the weight connecting layer j to layer k , and $f'(u^j)$ is the local gradient of the activation function. Each symbol is explicitly defined and linked back to the intuitive analogy from Layer 1. Following Sweller’s [1988] cognitive load theory, only essential symbols are introduced— δ and ∂ are the only new symbols in this chapter.

Layer 3: Code (Enactive). Learners verify their understanding by implementing the concept in code. The textbook [Xie, 2026] follows a progressive approach: first, a manual NumPy implementation of 3-layer backpropagation, followed by layer-by-layer comparison with PyTorch autograd values, and finally, automated PyTorch training on MNIST. This contrastive verification (DP5) is crucial: when learners see that their manual gradients match PyTorch’s automatic gradients at every layer, they gain confidence that their conceptual understanding is correct. This layer corresponds to Bruner’s [1966] enactive mode.

The three layers are applied in this specific order—intuition, then formula, then code—for every concept in the curriculum, from the M-P neuron in Chapter 1 through to the Transformer in Chapter 8.

2.4 Five Design Principles

We distill the triad into five transferable design principles (DP1–DP5) that can be applied independently of the specific curriculum:

Principle	Name	Description	Theory	Example
DP1	Minimal Symbol Principle	Introduce ≤ 2 new symbols per chapter; use common symbols (w over θ)	Sweller [1988]	Chapter 5 uses only δ and ∂
DP2	Visual-First Principle	Visualize every concept before formalizing it	Arcavi [2003]; Mayer [2001]	Watch convolution animation $\rightarrow$ convolution formula
DP3	Causal Chaining Principle	Every new concept emerges from the previous one’s limitation	Meyer & Land [2003]	M-P not differentiable $\rightarrow$ Sigmoid $\rightarrow$ need chain rule $\rightarrow \delta$ recursion
DP4	Progressive Abstraction	Concrete (NumPy) $\rightarrow$ Abstract (PyTorch autograd)	Bruner [1966]	Manual backpropagation $\rightarrow$ autograd
DP5	Verification-by-Contrast	Verify understanding through side-by-side comparison	Sfard [1991]	Manual gradient vs. autograd gradient

3. The Minimal Mathematical Path

This section presents the content-level innovation: a mathematically self-consistent path from the McCulloch-Pitts neuron to the Transformer.

3.1 Path Overview

The path is organized into three phases across the nine chapters of our textbook [Xie, 2026].

Phase 1: Foundation (Chapters 1–3). Chapter 1 introduces the M-P neuron, activation functions, and forward propagation. Chapter 2 provides the necessary mathematical tools—functions, vectors, matrices, derivatives, and the chain rule—each anchored to a specific neural network need. Chapter 3 introduces PyTorch’s Tensor, Autograd, and nn.Module abstractions progressively, starting from raw NumPy operations.

Phase 2: Core (Chapters 4–5). Chapter 4 covers loss functions, Softmax, and gradient descent variants, with 3D visualizations of loss landscapes. Chapter 5 is the conceptual heart of the curriculum: it derives the δ recursion formula, implements 3-layer backpropagation manually in NumPy, verifies against PyTorch autograd, and culminates in a complete MNIST classifier.

Phase 3: Advanced (Chapters 6–9). Chapter 6 introduces convolution as a solution to the parameter explosion problem. Chapter 7 covers training techniques (optimizers, regularization, BatchNorm). Chapter 8 traces the evolution from residual connections through attention mechanisms to a minimal Transformer implementation. Chapter 9 extends to large language models, covering autoregressive generation, scaling laws, KV cache acceleration, and LoRA fine-tuning.

3.2 Proof of Mathematical Self-Consistency

The core claim is that the path from the M-P neuron to the Transformer is not arbitrary but is a *topological sort of the mathematical dependency graph*. Each transition has a clear “breaking point”—the preceding concept’s limitation—that motivates the introduction of the next concept:

#	Transition	Limitation of Predecessor	How Successor Resolves It
1	M-P $\rightarrow$ Sigmoid	M-P is a step function, non-differentiable	Sigmoid is continuous and differentiable
2	Single neuron $\rightarrow$ Multi-layer	Single neuron can only solve linearly separable problems	Multi-layer + nonlinearity = universal approximator
3	Forward pass $\rightarrow$ Loss function	No way to quantify prediction quality	Loss function provides optimization objective
4	Loss function $\rightarrow$ Gradient descent	No analytical solution for millions of parameters	Iterative approximation via gradient information
5	Gradient descent $\rightarrow$ Backpropagation	Brute-force chain rule $O(L^2)$ is intractable	δ recursion reduces complexity to $O(L)$
6	Fully-connected $\rightarrow$ Convolution	Parameter explosion + lost spatial structure	Local connectivity + weight sharing
7		Degradation: deeper networks train worse	Skip connections provide gradient “highway”

#	Transition	Limitation of Predecessor	How Successor Resolves It
	Deep CNN → Residual connections		
8	CNN/RNN → Attention	Long-range dependencies + no parallelization	Self-attention: all-to-all + parallel
9	Attention → Transformer	No position information, no full encode-decode	Positional encoding + encoder-decoder + multi-head
10	Transformer → LLM	How to generate text from pretrained model	Autoregressive generation + KV cache + LoRA

Key insight: This path is the *shortest path* satisfying “every node depends only on predecessor nodes.” No concept in the path could be removed without breaking the chain, and no concept is introduced without a clearly articulated motivation from the preceding concept’s limitations.

3.3 Comparison with Existing Curricula

Dimension	Our Path	Typical University	D2L (Zhang)	DL from Scratch	Prince (2023)
Math self-consistency	★★★★★ chained	★★★ modular	★★★ topical	★★★★★ progressive	★★★★★ comprehensive
Intuition-first	★★★★★	★★	★★★	★★★★★	★★★
Code verification	★★★★★	★★★	★★★★★	★★★★★	★★★
Minimalist design	★★★★★	★★	★★★	★★★★★	★★
Entry threshold	Basic mathematics	Calculus + linear algebra	Python basics	Zero / basic	Undergraduate math
Content currency	2026 (incl. LLM)	Varies	2023 (incl. LLM)	2017 (no Transformer)	2023 (incl. diffusion)

4. Course Implementation

This section demonstrates how the framework is instantiated in the open-source textbook *Mathematics of AI Neural Networks* [Xie, 2026], available on GitHub.

4.1 Five-Stage Chapter Template

Every chapter in the textbook follows a standardized five-stage design that operationalizes the Intuition-First Triad:

- **Stage 1: Learning Objectives.** A checklist format that explicitly states what learners will be able to do after completing the chapter (“After this chapter, you can...”). This follows Wiggins & McTighe’s [2005] backward design principle—defining desired outcomes before designing activities.
- **Stage 2: Intuitive Introduction.** A 30-word analogy accompanied by a Matplotlib visualization. This corresponds to the Intuition Layer of the triad, building what Tall & Vinner [1981] call a “concept image” before formalization.
- **Stage 3: Mathematical Formulas.** Minimal notation with every symbol annotated and followed by a “human language” explanation. This corresponds to the Formula Layer, implementing the Minimal Symbol Principle (DP1).
- **Stage 4: Code Verification.** A NumPy manual implementation followed by a PyTorch automated version, with side-by-side comparison. This corresponds to the Code Layer, implementing Progressive Abstraction (DP4) and Verification-by-Contrast (DP5).
- **Stage 5: Key Insight.** A one-sentence summary of the core takeaway, connected causally to the next chapter (Causal Chaining Principle, DP3).

4.2 Chapter Mapping

Phase I: Foundation (Chapters 1–3)

Chapter	Core Content	Key Intuition	Key Code	Threshold Concept
Ch1	M-P neuron → Activation functions → Forward pass	“Neuron = weighted sum + activation”	NumPy M-P + 4 activations	Continuity from step to sigmoid
Ch2	Functions, vectors, matrices, derivatives, chain rule	“Each tool solves one network need”	Derivative viz + matrix multiply	Chain rule
Ch3	Tensor → Autograd → nn.Module	“Autograd computes gradients automatically”	Computational graph viz	Autograd’s graph mechanism

Phase II: Core (Chapters 4–5)

Chapter	Core Content	Key Intuition	Key Code	Threshold Concept
Ch4	Loss functions → Softmax →	“Gradient = direction downhill”		Convergence of GD

Chapter	Core Content	Key Intuition	Key Code	Threshold Concept
	Gradient descent variants		Loss landscape 3D viz	
Ch5	δ recursion $\rightarrow$ 3-layer BP $\rightarrow$ MNIST	“Error dominoes falling backward”	Manual BP vs autograd	δ recursion

Phase III: Advanced (Chapters 6–9)

Chapter	Core Content	Key Intuition	Key Code	Threshold Concept
Ch6	Convolution $\rightarrow$ Pooling $\rightarrow$ CNN on CIFAR-10	“Convolution = sliding window”	Edge detection + feature vis	Convolution
Ch7	Optimizer family $\rightarrow$ Regularization $\rightarrow$ BatchNorm	“Regularization = constraints on model”	Adam/SGD comparison + Dropout	Bias-variance tradeoff
Ch8	ResNet $\rightarrow$ Attention $\rightarrow$ Transformer	“Residual = gradient highway; Attention = search engine”	ResNet compare + mini Transformer	Self-attention
Ch9	Autoregressive $\rightarrow$ Scaling Law $\rightarrow$ KV Cache $\rightarrow$ LoRA	“LLM = next word predictor”	Sampling + inference acceleration	Emergent abilities

4.3 Detailed Pedagogical Case Studies

We present three case studies from the textbook [Xie, 2026] demonstrating the triad in action.

Case Study 1: Backpropagation (Chapter 5)

The learner pain point addressed here is the most frequently reported confusion in deep learning education: “Where does the gradient come from and where does it go?” Our pedagogical approach unfolds in three stages. First, the intuition layer introduces backpropagation through a domino metaphor—error signals travel backward through the network, with each neuron’s δ representing its “share of responsibility” for the final error. An animated visualization shows δ values propagating from the output layer back to the input layer, with color intensity proportional to error magnitude. Second, the formula layer presents the δ recursion: $\delta^i = \delta^k \cdot w^{kj} \cdot f'(u^j)$, where every symbol has a direct correspondence to the intuitive model. Third, the code layer walks learners through a three-step implementation: (1) a manual NumPy 3-layer backpropagation from scratch, (2) layer-by-layer gradient comparison with PyTorch autograd to verify correctness, and (3) automated PyTorch training on MNIST. The key insight is that backpropagation is simply an intelligent reuse of the chain rule—it

computes gradients in $O(L)$ time by caching and reusing intermediate results rather than recomputing from scratch at each layer.

Case Study 2: Residual Connections (Chapter 8)

The learner pain point is the counterintuitive observation that deeper networks can have *higher* training error. The intuition layer introduces residual connections as “a VIP express lane for gradients”—analogous to a junior employee reporting directly to the CEO rather than going through layers of management. The mathematics reveals why this works: $\partial L / \partial x = \partial L / \partial y \cdot (1 + \partial F / \partial x)$, where the identity term 1 ensures that gradients can bypass the learned transformation F entirely. The code implementation trains 20-layer networks with and without residual connections side by side, visualizing gradient distributions to show how skip connections prevent gradient vanishing. The key insight is that residual connections solve a gradient propagation problem, not a forward propagation problem.

Case Study 3: Attention’s Query-Key-Value (Chapter 8)

The learner pain point is the abstract nature of Q, K, V vectors. The intuition layer draws an analogy to search engines: Query is your search term, Key is the page title, Value is the page content. The mathematics formalizes this as $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d})V$, where the dot product QK^T measures relevance between queries and keys. The code implementation builds attention gradually: dot-product attention first, then scaled attention, then multi-head attention. The key insight is that attention is a *differentiable memory addressing* operation—it learns from data which elements to focus on.

5. Evaluation

We evaluate the framework through three complementary lenses: structural analysis, cognitive load analysis, and qualitative comparison.

5.1 Knowledge Path Completeness (DAG Analysis)

We constructed a knowledge dependency directed acyclic graph (DAG) for the complete textbook [Xie, 2026], consisting of 22 core concept nodes and 25+ dependency edges. The DAG exhibits three properties that confirm the path’s self-consistency:

1. **Full connectivity:** Every node has at least one incoming edge from a preceding chapter—no “orphan concepts” exist.
2. **Linear chain length:** The longest path from the M-P neuron (Chapter 1) to the Transformer (Chapter 8) has length 9, with no gaps.
3. **Monotonic depth:** Concepts in Chapter n only depend on concepts in chapters $\leq n$.

In contrast, typical university curricula exhibit modular DAGs where CNN and RNN modules exist in parallel without causal connections, leaving learners unable to trace the mathematical motivation for architectural innovations like attention mechanisms.

5.2 Cognitive Load Analysis

We counted the number of new mathematical symbols introduced per chapter in our textbook and compared with three major textbooks. Our curriculum introduces 2–3 new symbols per chapter (e.g., Chapter 5 introduces only δ and ∂), while comparable textbooks introduce 5–8 new symbols per chapter. According to Sweller’s [1988] cognitive load theory, this gentler symbol introduction curve should reduce extraneous cognitive load. A full statistical analysis following established CLT measurement frameworks [Paas et al., 2003] is ongoing and will be reported in the final version.

5.3 Qualitative Comparison with Existing Frameworks

Framework	Common Ground	Our Differentiator
TinyTorch [Reddi, 2026]	Build from principles	We focus on conceptual path, not systems implementation
CNN Explainer [Wang et al., 2021]	Visualization-driven	We cover the full DL path, not a single architecture
D2L [Zhang et al., 2023]	Code-driven	Our intuition→formula→code order reverses theirs
DL from Scratch [斎藤, 2017]	Build from scratch	We extend to Transformer+LLM with systematic theory

5.4 Limitations

We acknowledge several limitations. First, the current evaluation is analytical rather than empirical—a controlled study comparing intuition-first sequencing with traditional approaches is planned. Second, the framework assumes basic mathematics proficiency; learners below this threshold may need additional support. Third, the current implementation is primarily in Chinese, with English and PDF versions available on GitHub; cross-linguistic validation across both versions is needed.

6. Discussion and Future Work

6.1 Generalizability

The Intuition-First Triad and its five design principles are deliberately framework-agnostic. They can potentially be applied to other AI subfields (NLP curricula, reinforcement learning, computer vision), other STEM disciplines (physics, statistics, signal processing), and different educational levels (K-12 AI education with a simplified intuition layer, or graduate courses with a deepened formula layer).

6.2 Tools and Infrastructure

We plan to develop an interactive visualization toolkit including backpropagation animations, loss landscape 3D viewers, and attention heatmap explorers. The companion repository [Xie, 2026] already

provides Jupyter Notebooks and plans to add Google Colab integration for one-click environment setup. A knowledge path navigator dashboard is also under development.

6.3 Future Research Directions

Future work includes controlled A/B experiments comparing intuition-first and traditional sequencing, adaptive learning paths that dynamically adjust depth based on learner background, cross-cultural validation between the Chinese and English versions of the textbook, and LLM-assisted teaching tools that extend the textbook’s “little elf” analogy character into an interactive AI tutor.

7. Conclusion

This paper presented a unified curriculum design framework for deep learning education that bridges the persistent “math-code gap.” The **Intuition-First Triad** provides a pedagogical methodology grounded in established cognitive and educational theories, while the **Minimal Mathematical Path** offers a mathematically self-consistent knowledge trajectory from the M-P neuron to the Transformer—a topological sort of deep learning’s dependency graph. The five design principles (DP1–DP5) are deliberately transferable, enabling other educators to adopt, adapt, and extend the framework.

The framework is instantiated in the open-source textbook *Mathematics of AI Neural Networks* [Xie, 2026] (available on GitHub), which provides a complete 9-chapter implementation with 35+ code files and 35+ visualizations. We conclude with a call to the AI education community: quality in AI education requires not just *what* we teach, but *why we teach it in that order*. A causally chained, intuition-first path is not merely a pedagogical convenience—it is the foundation for genuine conceptual understanding.

References

Deep Learning Textbooks

[1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. [2] Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). Dive into Deep Learning. *arXiv:2106.11342*. [3] Prince, S. J. D. (2023). *Understanding Deep Learning*. MIT Press. [4] 斎藤康毅. (2017). ゼロから作る*Deep Learning*. O’Reilly Japan. (English: *Deep Learning from Scratch*)

AI/ML Education Research

[5] Sulmont, E., Patitsas, E., & Cooperstock, J. R. (2019). Can You Teach Me To Machine Learn? *SIGCSE ’19*, 948–954. <https://doi.org/10.1145/3287324.3287392> [6] Reddi, V. J. (2026). TinyTorch: Building Machine Learning Systems from First Principles. *arXiv:2601.19107*.

Cognitive Science & Mathematics Education

[7] Bruner, J. S. (1966). *Toward a Theory of Instruction*. Harvard University Press. [8] Sweller, J. (1988). Cognitive Load During Problem Solving: Effects on Learning. *Cognitive Science*, 12(2), 257–

285. https://doi.org/10.1207/s15516709cog1202_4 [9] Mayer, R. E. (2001). *Multimedia Learning*. Cambridge University Press. [10] Tall, D., & Vinner, S. (1981). Concept Image and Concept Definition in Mathematics. *Educational Studies in Mathematics*, 12(2), 151–169. <https://doi.org/10.1007/BF00305619> [11] Arcavi, A. (2003). The Role of Visual Representations in the Learning of Mathematics. *Educational Studies in Mathematics*, 52(3), 215–241. [12] Paivio, A. (1986). *Mental Representations: A Dual Coding Approach*. Oxford University Press.

Curriculum Design

[13] Wiggins, G., & McTighe, J. (2005). *Understanding by Design (2nd ed.)*. ASCD. [14] Meyer, J., & Land, R. (2003). Threshold Concepts and Troublesome Knowledge. *Improving Student Learning*, 10, 412–424. [15] Villanueva, A. N., & Simske, S. J. (2023). Functional Analytics for Document Ordering for Curriculum Development. *arXiv:2312.09457*. [16] Sfard, A. (1991). On the Dual Nature of Mathematical Conceptions. *Educational Studies in Mathematics*, 22(1), 1–36.

Visualization Tools

[17] Wang, Z. J., et al. (2021). CNN Explainer: Learning Convolutional Neural Networks with Interactive Visualization. *IEEE Transactions on Visualization and Computer Graphics*, 27(2), 1396–1406. [18] Smilkov, D., et al. (2017). TensorFlow Playground. <https://playground.tensorflow.org>

Our Textbook

[19] Xie, Fujin [xiefujin]. (2026). *Mathematics of AI Neural Networks*. GitHub repository: <https://github.com/kandada/math-ai-nn>. Chinese version: <https://github.com/kandada/math-ai-nn/tree/main/zh>, English translation: <https://github.com/kandada/math-ai-nn/tree/main/en>, PDF: <https://github.com/kandada/math-ai-nn/tree/main/pdf>.

Other

[20] Paas, F., Tuovinen, J. E., Tabbers, H., & Van Gerven, P. W. M. (2003). Cognitive Load Measurement as a Means to Advance Cognitive Load Theory. *Educational Psychologist*, 38(1), 63–71.